

Assignment 6 - CI/CD & Testing

Start Assignment

- Due Aug 18 by 11:59pm
- Points 17
- Submitting a text entry box or a website url

This is the final and most comprehensive assignment. You will deploy your entire sentiment analysis system—the FastAPI backend and the Streamlit monitoring dashboard—onto a live cloud server (AWS EC2). You will also implement a professional CI/CD (Continuous Integration/Continuous Deployment) pipeline using GitHub Actions to automate testing and linting, ensuring code quality before deployment.

Due Date: 18 August 2025

System Architecture on EC2

- **AWS EC2 Instance:** A single virtual server running a Linux distribution (e.g., Ubuntu).
- **Docker:** You will install Docker on the EC2 instance to run your application.
- **Services:** The FastAPI backend and Streamlit dashboard will run as two separate **Docker containers in detached mode (`-d`)** on the same EC2 instance.

Part 1: Preparing the Application for Production

In this part, you will enhance your existing project with tests to ensure reliability.

Tasks:

1. Create Test Files:

- **API Testing (`api/test_api.py`):** Write tests for your FastAPI application using `pytest`. You must test:

- The `/predict` endpoint with both a positive and a negative example.
- That the endpoint correctly handles requests with missing or malformed data.
- **Dashboard Testing** (`monitoring/test_dashboard.py`): Write at least one simple test for your Streamlit application to ensure it can launch without errors.

Part 2: CI/CD with GitHub Actions

You will automate code quality checks for every pull request made to your `main` branch.

Tasks:

1. Set up Git Branches:

- CREATE A NEW REPOSITORY FOR THIS ASSIGNMENT
- All your work for this assignment must be done on a `dev` branch. The `main` branch should be protected.

2. Create the GitHub Actions Workflow:

- In your repository, create the directory `.github/workflows/`.
- Inside, create a YAML file (e.g., `ci.yml`). This workflow must:
 - **Trigger:** Be triggered automatically whenever a pull request is opened or updated against the `main` branch.
 - **Jobs:** Define a job that runs on an Ubuntu runner.
 - **Steps:** The job must perform the following steps in order:
 1. Check out the code from the repository.
 2. Set up a specific version of Python.
 3. Install all project dependencies from `requirements.txt`
 4. **Linting:** Run a linter like `flake8` or `ruff` against your Python code to check for style issues.

5. **Testing:** Run your entire test suite using `pytest`.

Part 3: Deployment to AWS EC2

This is the manual deployment process you will document.

Tasks:

1. Launch and Configure EC2 Instance:

- Launch a new `t2.micro` EC2 instance with Ubuntu.
- Configure its **Security Group** to allow incoming traffic on:
 - **Port 22 (SSH)** from your IP address for access.
 - **Port 8000 (FastAPI)** from anywhere.
 - **Port 8501 (Streamlit)** from anywhere.
- Connect to your EC2 instance using SSH.

2. Set up the Server Environment:

- On the EC2 instance, install **Docker** and **Git**.

3. Deploy the Application:

- Clone your GitHub repository onto the EC2 instance.
- Create a shared Docker volume for the logs.
- Build the Docker images for both the `api` and `monitoring` services.
- Run both containers in **detached mode** (`-d`), ensuring they are connected to the shared volume.

Part 4: Documentation (The README.md)

Your `README.md` is the final, most critical piece. It must serve as a complete operational manual for your project.

Tasks:

- **Update the README.md to include:**

1. **Project Architecture:** A clear description of the final architecture, including the FastAPI service, the Streamlit dashboard, the CI/CD pipeline, and the deployment on EC2.
2. **Local Development:** Instructions on how to build and run the project locally using Docker.
3. **Manual Deployment Guide (Very Detailed):** A step-by-step guide for a new developer to deploy this project from scratch. This must include:
 - How to launch and configure the EC2 instance and its security group.
 - All the commands needed to install dependencies (Git, Docker) on the server.
 - The exact `docker` commands to create the volume, build the images, and run the two containers in detached mode.

Submission

- Create a pull request from your `dev` branch to your `main` branch on GitHub. The PR should show the status of your GitHub Actions checks. **Do not merge it until after grading.**
- Submit the URL to the **pull request** on your public GitHub repository. This will allow me (the instructor) to see both your code and the results of the automated CI/CD pipeline.
- Use the AWS sandbox for this assignment. I will not check your EC2 instance since it will not persist once you close it, instead I will replicate the steps you mentioned and clone it on my end to grade your work